COMP90077
Lecture 11
Online Algorithms

William Umboh

University of Melbourne

Caution

- Slides are designed as presentation aid to go along with lecture, not as study material
- Slides not designed to make sense on their own

Assignment 2

- Due: Saturday 30 May 17:00
- Problems conceptually harder than A1
- Take a look ASAP if you have not already
- Problems usually require subconscious processing
- Spend half an hour to load the problems so that your subconscious can work on it in the background
- Any partners?
- Feel free to approach me over email/consultation if you are not sure of level of details required in responses (proofs, reflection, etc)

Poll

- How well do you understand LPs?
- Rate the effectiveness of the teaching of LPs (lectures, notes, tutorials)
- What additional resources would you find helpful?

PollEv
https://pollev.com/surveys/m4TuzPOuKhH9Dm2FS2VPu/respond

Today

- Last season:
  - Linear Programming aka Continuous/Fractional Relaxation
  - Applications to Approximation Algorithms
- ✨Online Algorithms✨
  - ⛷️Rent or Buy 🚃
  - Bipartite Matching
  - Set Cover

Online Algorithms

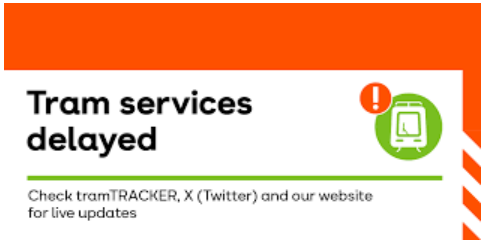
- Traditional computational model
  - Given entire input, do some computation, output solution
- In practice, often need to make decisions over time without complete knowledge of future input
  - **Uber**: allocating drivers to passengers
  - **Cloud computing**: assigning jobs to machines
  - **Ad allocation**: match users to ads

Rent or Buy Problem

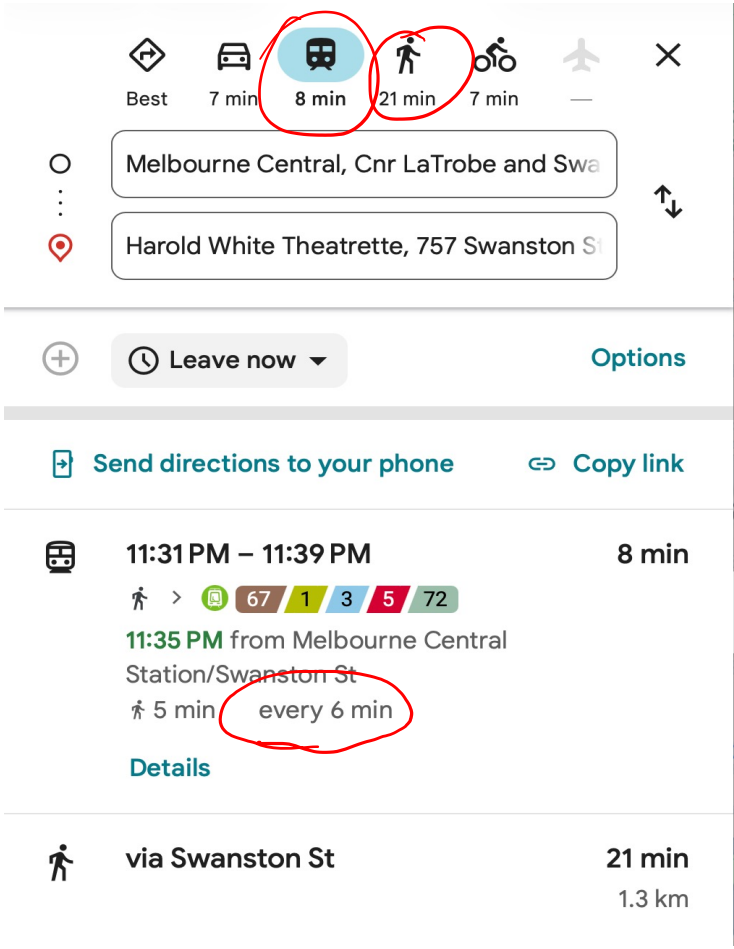
- Simplest online problem
- Aka “Ski Rental Problem”
- But this is Melbourne
- So “Tram Waiting Problem”

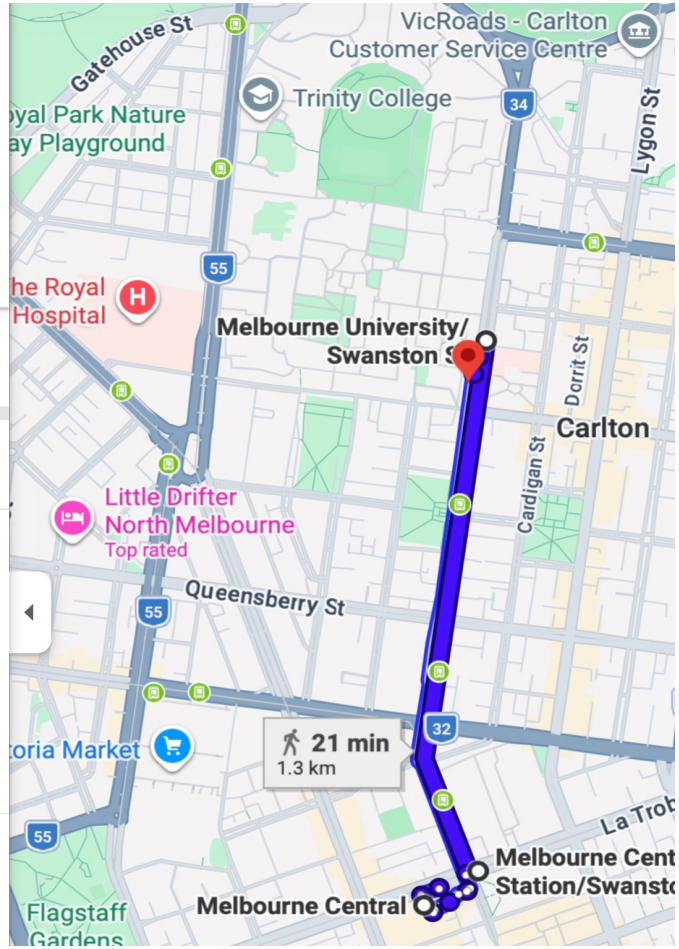
Tram Waiting Problem

- You are at Melbourne Central and want to get to lecture
- Walking takes 21 mins
- Tram takes 8 mins
- But...



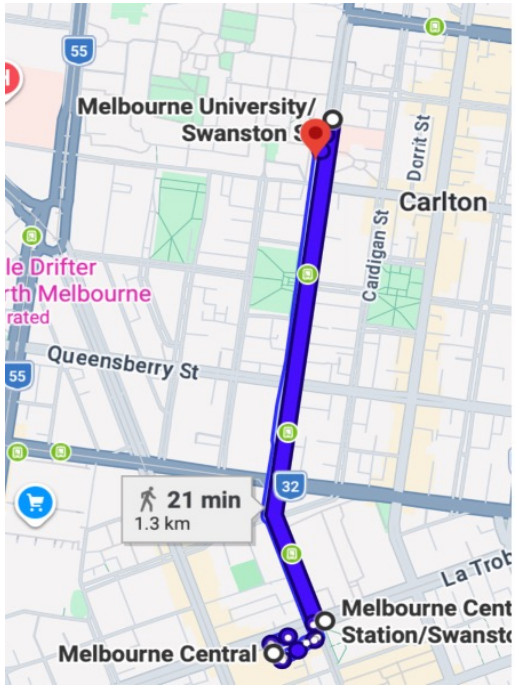
- Not sure when tram will arrive
- Q: When to give up waiting and walk instead?





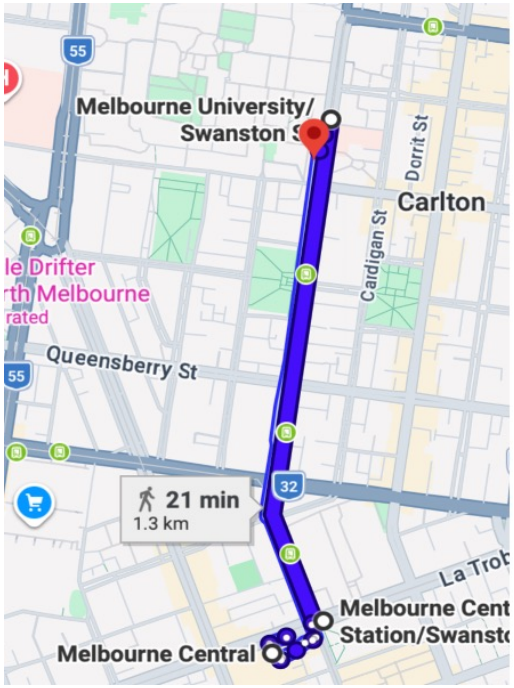
Tram Waiting Problem

- Let's make things simpler
- Suppose happy once on tram or at destination
- At each timestep (1 min per step),
  - if tram has arrived, 😁
  - if no tram 😕, decide whether to walk or wait 1 more min
  - once you start walking, you must keep walking
- Goal: Minimize waiting time + walking time
- Equivalently
  - Determine threshold t such that after waiting for t mins, we walk for 21 mins



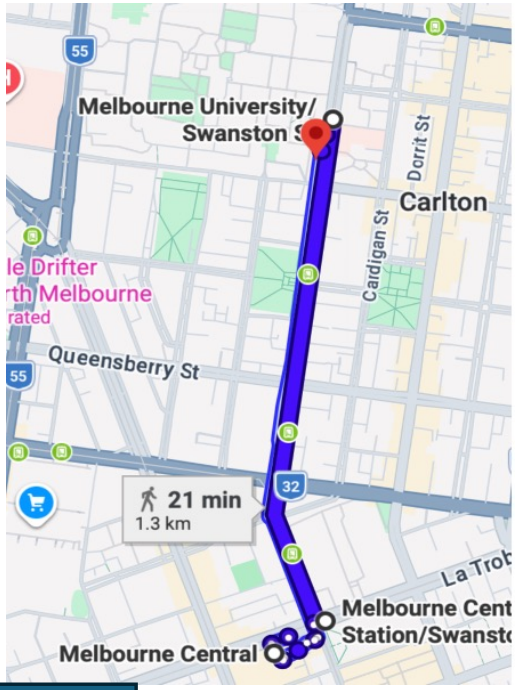
Tram Waiting Problem

- Problem: Determine threshold t such that after waiting for t mins, we walk for 21 min
- Goal: Minimize waiting time + walking time
- Suppose we know tram will arrive after x minutes
  - What should t be?
- Then:
  - $t^* = x$ if $x \leq 21$ for a cost of x
  - $t^* = 0$ if $x > 21$ for a cost of 21
  - OPT = $\min\{x, 21\}$
- Offline setting/algorithm: have full information



Tram Waiting Problem - Online

- Problem: Determine threshold t such that after waiting for t mins, we walk for 21 min
- Goal: Minimize waiting time + walking time
- Suppose tram will arrive after x minutes
  - No idea what x is
  - Can be 5, 22, 50
  - Or heat death of universe 💀
  - After waiting for w mins, we learn if $x = w$ or $x > w$
  - What is a good choice of t?



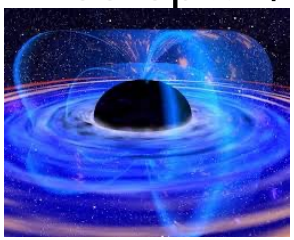
How do we measure "goodness" of online algorithm?

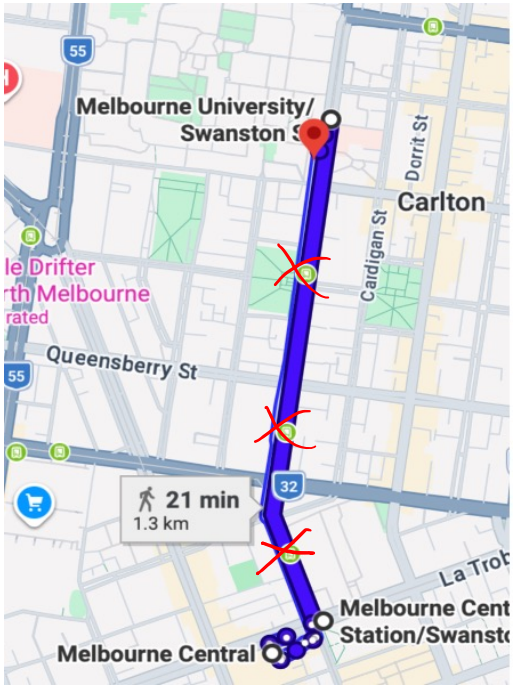
Benchmark: Competitive Analysis

- Worst-Case Analysis for Online Algorithms
- Competitive ratio: ALG/OPT where OPT is the optimal solution with full knowledge (aka offline optimal)
- Minimization: Algorithm is α-competitive if cost $\leq \alpha$OPT on every instance
- Maximization: Algorithm is α-competitive if value $\geq \alpha$OPT on every instance
- Observe:
  - No historical data
  - No machine learning
  - Future is adversarial

Tram Waiting Problem - Online

- Goal
  - Determine threshold t such that after waiting for t mins, we walk for 21 mins
  - Minimize waiting time + walking time
- Suppose tram will arrive after x minutes (timestep $x + 1$)
  - No idea what x is
  - Can be 5, 22, 50
  - Or heat death of universe 💀
  - After waiting for w mins, we learn if $x = w$ or $x > w$
  - How to choose t that is competitive for every x?
- Intuitively: choose $t = 21$.





General Tram Waiting Problem - Online

- Goal
  - Determine threshold t such that after waiting for t mins, we walk for W mins
  - Minimize waiting time + walking time
- Greedy Algorithm: $t = W$
  - Break-even, balances waiting time and walking time
- Greedy is 2-competitive (for every x, Greedy $\leq$ 2OPT)

💡 Key Idea for Charging:
Greedy walks for W mins only after waiting for W mins

Proof 1: Case analysis

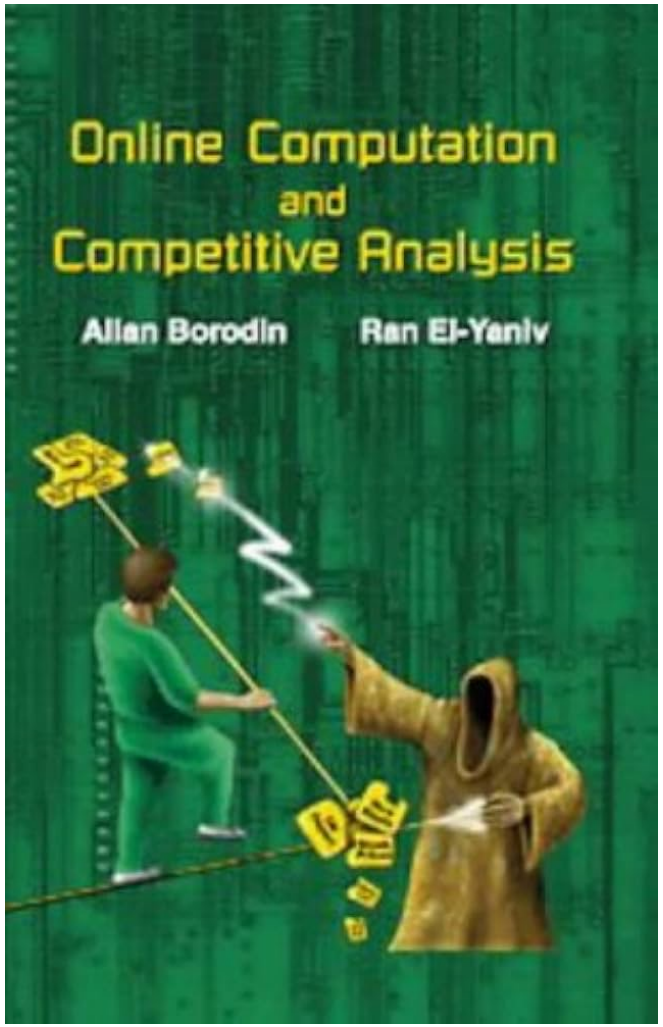
- Greedy = x if $x \leq W$ and
  $2W$ if $x > W$
- OPT $= \min\{x, W\} = x$ if $x \leq W$ and
  W if $x > W$

Proof 2: Charging argument

- Greedy $\leq 2x$ waiting time
  - walking time $\leq$ waiting time
  - waiting time $\leq x$ $\min\{x, W\} = OPT$
- ~~OPT $= \min\{x, W\} \geq x$~~

Is there a better online algorithm?

- Prove lower bounds via adversarial arguments
- Adversary constructs next input knowing the algorithm's actions so far and how the algorithm will react to possible next inputs
- Think of it as a 2-player game
- At each timestep (1 min per step),
  - Adversary decides if tram has arrived
  - If tram has arrived, game ends
  - Else, algorithm whether to walk or wait 1 more min
- Adversary's goal is to maximize ALG/OPT
- What should the adversary do?
- Intuitively, tram arrives right after algorithm walks



Lower Bound of 2

Thm. Every deterministic algorithm has competitive ratio at least 2

$(\exists t \text{ s.t. } \frac{ALG}{OPT} \geq 2)$

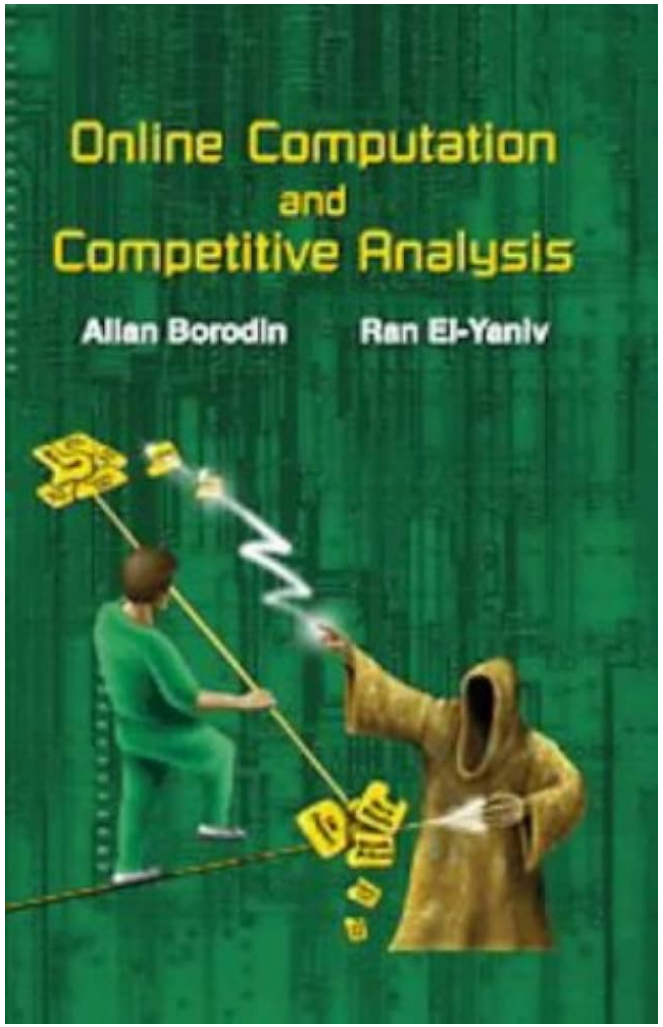
Proof.

- Suppose after waiting for t mins, algorithm walks for W mins
- Adversary sets $x = t$ (tram arrives at $t + 1$)
- Algorithm walks after waiting for t minutes
  - ALG = t + W
- OPT = $\min\{t, W\} \leq (t + \text{W})/2$
- QED

$\min\{a, b\} \leq \frac{a+b}{2}$

Rent or Buy - Summary

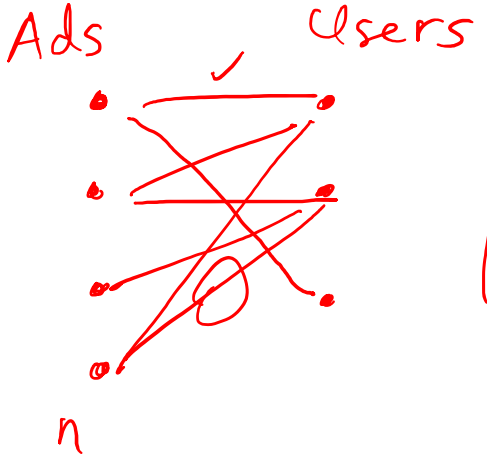
- Deterministic algorithms
  - Greedy is 2-competitive
  - No algorithm can do strictly better
- Randomized algorithms
  - Can do better than deterministic algorithms
  - Intuition: Adversary does not know exactly what algorithm will do next
  - Expected competitive ratio $\frac{e}{e-1} = 1.58 \ldots$



Rent or Buy Problems are Everywhere

- Fundamental problem in online algorithms
- Occurs as subproblem in other online problems
  - Rent action = cheap but short-term
  - Buy action = expensive but long-term
- Lots of practical applications
- Equipment maintenance:
  - Have old equipment that needs weekly maintenance
  - New equipment expensive but does not require maintenance

Online Bipartite Matching



$\theta = 30\%$

$\frac{n}{3}$

- Initially:
  - Given left-hand side of bipartite graph
  - Matching $M = \emptyset$
- At each time step:
  - A right-hand side vertex v arrives and its edges are revealed
  - Decide whether to add one of v's edges to M
  - M must be a matching at all times

RHS vertices unknown.

- Goal: Maximize $|M|$
- Irrevocable decisions:
  - Cannot remove edges from M
  - Cannot add previous edges
- Applications: Ad allocations

Any ideas?

Greedy Strikes Back

- Greedy = Add any edge that we can

Thm. Greedy is (1/2)-competitive

Pf.

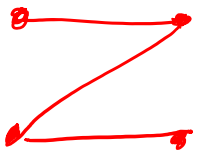
- Greedy matching = maximal matching
- Maximal matching = (1/2)-approximation

Can we do better?

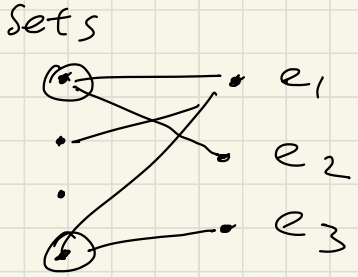
Any ideas?

Thm. Every deterministic algorithm has competitive ratio ≥ 2.

Pf.



Online Set Cover



Initially, n elements

m sets $S_1, \ldots, S_m$

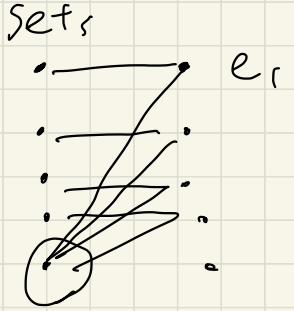
At each timestep,

- Element e_j needs to be covered
- Buys a set if it needs to

Unknown set system setting (Deterministic algorithms) $\Omega(m)$

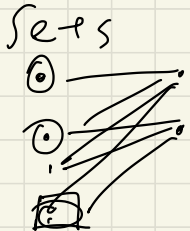
Only sets are known.

Only given LHS of bipartite graph



$ALG = m$

$OPT = 1$



Thm There is a randomized $O(\log n \cdot \log m)$-competitive polynomial time algorithm.

Approach

① Solve the online fractional set cover problem
$O(\log m)$-comp.

② Round the fractional solution online.
$O(\log n)$ - rounding.

Online Fractional Set Cover

Initially, n elements

m sets $S_1, \ldots, S_m$

variables $x_1, \ldots, x_m = 0$

At each timestep,

- Element e_j needs to be covered
- Raise variables $x_1, \ldots, x_m$

$$\text{s.t.} \sum_{S_i \ni e_j} x_i \geq 1$$

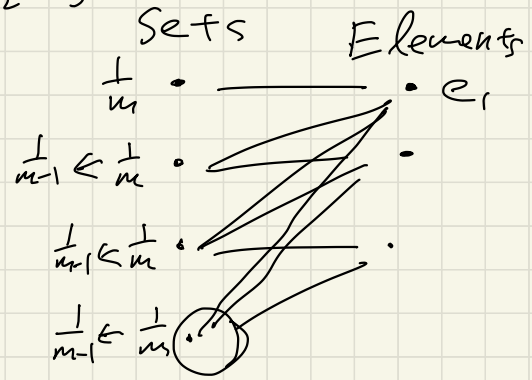
Competitive ratio: $\frac{ALG}{OPT}$ ← LP optimal

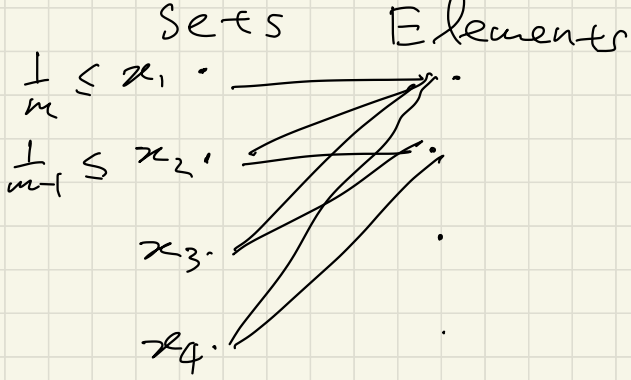
Lower Bounds against Fractional Algorithms

Thm Every fractional alg has comp. ratio $\Omega(\log m)$

$H_m = \Omega(\log m)$

$1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{m}$





Thm There is a $O(\log m)$-comp. algorithm for Online Fractional Set Cover.

Algorithm (Multiplicative Weights Update)

Initialize $x_i = \frac{1}{m}$

while $\sum_{S_i \ni e_j} x_i < 1$:

for every $S_i \ni e_j$:

$x_i \leftarrow 2x_i$

$y_j \leftarrow y_j + 1$

Claim $\sum_i x_i \le \sum_j y_j$

Pf In each iteration of while loop,

$\Delta LHS \le \Delta RHS$.

$\Delta LHS = \sum_{S_i \ni e_j} x_i < 1$

$\Delta RHS = 1$ □

Dual constraints: $\forall S_i \quad \sum_{e_j \in S_i} y_j \leq 1$

Claim $\underbrace{\sum_{e_j \in S_i} y_j}_{\text{\# of times } x_i \text{ is doubled.}} \leq O(\log m)$

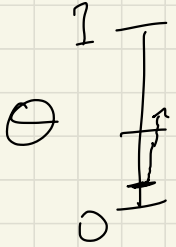
$\leq O(\log m)$.

Since $x_i = \frac{1}{m}$ initially

and once $x_i = 1$ covers all $e \in S_i$

$\Rightarrow x_i \leq 1$

Rounding: Pick a threshold $\Theta \in [0, 1]$ uniformly at random.



Buy S_i once $x_i (\log n) \geq \Theta$